LangGraph

A Complete Developer Guide

Build stateful, multi-agent AI applications with graph-based orchestration. This guide covers core concepts, architecture, practical patterns, and production-ready code examples.

Version	1.0
Framework	LangGraph (LangChain Ecosystem)
Language	Python 3.9+
Difficulty	Intermediate – Advanced

1. Introduction to LangGraph

LangGraph is an open-source library built on top of LangChain that enables developers to create stateful, multi-actor applications powered by large language models (LLMs). It models agent workflows as directed graphs — or even cyclic graphs — giving you fine-grained control over execution flow, state persistence, and multi-agent coordination.

Why LangGraph?

- Traditional LLM chains are linear and stateless — they cannot loop or branch based on runtime conditions.
- LangGraph introduces **cycles**, **branches**, and **persistent state** so agents can reason, retry, and collaborate.
- It integrates natively with LangChain tools, memory stores, and model providers.
- It provides first-class support for human-in-the-loop workflows and streaming.

Core Value Propositions

Feature	Description
Stateful Graphs	Maintain state across multiple LLM calls automatically
Cyclic Execution	Agents can loop until a condition is met (e.g., task complete)
Multi-Agent	Coordinate multiple specialized agents within one graph
Human-in-loop	Pause execution and resume after human approval
Streaming	Stream tokens and intermediate steps in real time
Checkpointing	Persist graph state to resume interrupted workflows

2. Installation & Setup

Install via pip

```
# Install LangGraph and LangChain core pip install langgraph langchain langchain-openai # Optional: Install with checkpointing support pip install langgraph[checkpoint-postgres]
```

Environment Variables

```
import os os.environ['OPENAI_API_KEY'] = 'sk-...' os.environ['LANGCHAIN_TRACING_V2'] = 'true' # LangSmith tracing os.environ['LANGCHAIN_API_KEY'] = 'ls-...'
```

Tip: Use a .env file and load it with python-dotenv for cleaner secret management.

3. Core Concepts

3.1 Nodes

A **node** is a Python function (or a runnable) that takes the current graph state as input and returns an updated state. Nodes represent discrete units of work — an LLM call, a tool invocation, a validation step, etc.

```
from langgraph.graph import StateGraph from typing import TypedDict class AgentState(TypedDict): messages: list next_step: str def call_llm(state: AgentState) -> AgentState: # Call the model and update state response = llm.invoke(state['messages']) return {'messages': state['messages'] + [response]}
```

3.2 Edges

Edges define how execution flows between nodes. LangGraph supports three edge types:

Normal Edge — Always transitions from node A to node B.

Conditional Edge — Chooses the next node based on a function that inspects state.

Entry/End Points — Special markers for where the graph starts and terminates.

3.3 State

The **state** is a typed dictionary (TypedDict or Pydantic model) shared across all nodes in the graph. Each node receives the full state and returns a partial or full update. LangGraph merges updates automatically using **reducers**.

```
from typing import Annotated from operator import add class State(TypedDict): # 'add' reducer: new messages are appended messages: Annotated[list, add] iteration_count: int final_answer: str
```

4. Building Your First Graph

Below is a minimal but complete example of a ReAct-style agent using LangGraph. The agent calls an LLM, optionally invokes tools, and loops until the LLM decides to stop.

```
from langchain_openai import ChatOpenAI from langchain_core.messages import HumanMessage, AIMessage from langgraph.graph import StateGraph, END from langgraph.prebuilt import ToolNode from typing import TypedDict, Annotated from operator import add # 1. Define state class AgentState(TypedDict): messages: Annotated[list, add] # 2. Define model + tools llm = ChatOpenAI(model='gpt-4o', temperature=0) tools = [search_tool, calculator_tool] # your tools llm_with_tools = llm.bind_tools(tools) # 3. Define nodes def agent_node(state: AgentState): response = llm_with_tools.invoke(state['messages']) return {'messages': [response]} tool_node = ToolNode(tools) # 4. Router: should we call a tool or stop? def should_continue(state: AgentState): last = state['messages'][-1] if hasattr(last, 'tool_calls') and last.tool_calls: return 'tools' return END # 5. Build the graph graph = StateGraph(AgentState) graph.add_node('agent', agent_node) graph.add_node('tools', tool_node) graph.set_entry_point('agent') graph.add_conditional_edges('agent', should_continue) graph.add_edge('tools', 'agent') # loop back app = graph.compile() # 6. Run result = app.invoke({'messages': [HumanMessage('What is 25 * 48?')]}) print(result['messages'][-1].content)
```

5. Checkpointing & Persistence

Checkpointing allows LangGraph to save graph state after every node execution. This enables you to resume interrupted workflows, implement human-in-the-loop approval steps, and replay past runs for debugging.

In-Memory Checkpointer (development)

```
from langgraph.checkpoint.memory import MemorySaver memory = MemorySaver() app =
graph.compile(checkpointer=memory) # Use a thread_id to maintain separate conversation
sessions config = {'configurable': {'thread_id': 'user-42'}} result =
app.invoke({'messages': [HumanMessage('Hello')]}, config=config)
```

PostgreSQL Checkpointer (production)

```
from langgraph.checkpoint.postgres import PostgresSaver DB_URI =
'postgresql://user:pass@localhost/langgraph' with PostgresSaver.from_conn_string(DB_URI) as
checkpointer: checkpointer.setup() # create tables app =
graph.compile(checkpointer=checkpointer) result = app.invoke(input_state,
config={'configurable': {'thread_id': 'session-1'}})
```

Human-in-the-Loop with Interrupts

```
from langgraph.graph import StateGraph, END # Compile with interrupt_before a sensitive node
app = graph.compile( checkpointer=memory, interrupt_before=['execute_action'] # pause here )
# First run — halts before 'execute_action' app.invoke(initial_state, config=config) # Human
reviews state, then resumes app.invoke(None, config=config) # None = resume from checkpoint
```

6. Multi-Agent Patterns

LangGraph excels at orchestrating multiple specialized agents. Two common patterns are the **Supervisor** pattern and the **Swarm** pattern.

6.1 Supervisor Pattern

A central supervisor node decides which specialized worker agent to call next. Workers return results to the supervisor, which routes to the next worker or ends.

```
WORKER_AGENTS = ['researcher', 'coder', 'critic'] def supervisor(state): # Ask the LLM which agent to call next decision = supervisor_llm.invoke(state['messages']) return {'next': decision.next_agent} def route_to_worker(state): return state['next'] # 'researcher' | 'coder' | 'critic' | '__end__' graph.add_conditional_edges('supervisor', route_to_worker, {w: w for w in WORKER_AGENTS} | {'__end__': END})
```

6.2 Subgraphs

Complex agents can be encapsulated as subgraphs and embedded as nodes in a parent graph, enabling hierarchical decomposition of tasks.

```
# Compile a subgraph research_subgraph = research_graph.compile() # Use it as a node in the parent graph parent_graph.add_node('research_agent', research_subgraph) parent_graph.add_edge('start', 'research_agent') parent_graph.add_edge('research_agent', 'writer_agent')
```

7. Streaming

LangGraph supports multiple streaming modes for real-time output. You can stream final tokens, intermediate node updates, or both simultaneously.

Stream Node Updates

```
for chunk in app.stream({'messages': [HumanMessage('Explain quantum computing')]}): for node_name, node_output in chunk.items(): print(f'Node: {node_name}') print(node_output) print('---')
```

Stream LLM Tokens

```
async for event in app.astream_events( {'messages': [HumanMessage('Write a poem')]}, version='v2' ): if event['event'] == 'on_chat_model_stream': token = event['data']['chunk'].content print(token, end='', flush=True)
```

8. LangGraph Platform & Cloud

LangGraph Platform (formerly LangGraph Cloud) provides managed infrastructure for deploying, scaling, and monitoring LangGraph applications in production.

Key Features

- Persistent task queue for long-running agents
- Automatic horizontal scaling and load balancing
- Built-in state and thread management
- LangSmith integration for tracing and evaluation
- REST API + Python/JS SDK for client integration

Deployment with LangGraph CLI

```
# Install the CLI pip install langgraph-cli # Initialize project langgraph new my-agent # Local dev server langgraph dev # Deploy to LangGraph Cloud langgraph deploy
```

9. Best Practices

✦ Type your state strictly

Always use TypedDict or Pydantic for state. This catches bugs early and improves IDE support.

✦ Keep nodes small & focused

Each node should do one thing. This makes graphs easier to test, debug, and reuse.

✦ Use reducers for lists

Annotate list fields with a reducer (e.g., operator.add) to avoid overwriting previous messages.

✦ Enable tracing in development

Set LANGCHAIN_TRACING_V2=true to get full execution traces in LangSmith for free.

✦ Use MemorySaver locally, Postgres in prod

MemorySaver is in-process and ephemeral; PostgresSaver survives restarts.

✦ Limit max iterations

Always add a recursion_limit to prevent infinite loops: app.invoke(state, config={'recursion_limit': 25}).

✦ Test nodes independently

Since nodes are plain Python functions, unit-test them directly before wiring into a graph.

✦ Use interrupt_before for sensitive operations

Pause before irreversible actions (sending emails, executing code) to allow human review.

10. Quick Reference Cheat Sheet

Task	Code
Create graph	graph = StateGraph(MyState)
Add node	graph.add_node('name', fn)
Set entry	graph.set_entry_point('name')
Add edge	graph.add_edge('a', 'b')
Conditional edge	graph.add_conditional_edges('a', router_fn)
Compile	app = graph.compile()
Compile + memory	app = graph.compile(checkpointer=MemorySaver())
Invoke	app.invoke({'messages': [...]})
Stream	for chunk in app.stream(state): ...
Async invoke	await app.ainvoke(state)
Get state	app.get_state(config)
Update state	app.update_state(config, values)
Resume	app.invoke(None, config=config)
List threads	app.get_state_history(config)

Resources: langgraph.dev | github.com/langchain-ai/langgraph | python.langchain.com/docs/langgraph | LangSmith: smith.langchain.com